

MODULE 3.1: SYMMETRIC KEY ALGORITHM

Contents

Symmetric Key Algorithm.....	1
Types of Symmetric Key Algorithms	1
Modes of Operation in Block Cipher	2
Confusion and Diffusion.....	3
Confusion	3
Diffusion	4
Stream Cipher vs. Block Cipher	5
Stream Ciphers	5
Block Ciphers	6
The Feistel Cipher Structure	7

Symmetric Key Algorithm

- A Symmetric Key Algorithm is an encryption technique where the same secret key is used for both encryption and decryption of data.
- The sender encrypts the plaintext using a secret key, and the receiver uses the same key to decrypt the ciphertext back into plaintext.
- It is called “symmetric” because the same key is shared between communicating parties.

Overview of Symmetric Key Cryptography

Symmetric key cryptography is one of the oldest and fastest encryption methods.

It provides:

- Confidentiality of data
- Fast encryption and decryption
- Efficient processing for large files and networks

However, the major challenge is secure key distribution because both sender and receiver must know the same secret key.

Characteristics

1. Uses a single shared key
2. Faster than asymmetric encryption
3. Suitable for bulk data encryption
4. Requires secure key sharing
5. Less computationally expensive

Types of Symmetric Key Algorithms

Symmetric algorithms are mainly divided into two categories:

1. **Block Cipher:** A Block Cipher encrypts data in fixed-size blocks. Example: AES, DES, Blowfish.

Suppose block size is 64 bits:

- Plaintext is divided into 64-bit blocks
- Each block is encrypted separately

Features

- More secure
- Used in modern systems
- Supports different modes of operation

2. Stream Cipher: A Stream Cipher encrypts data one bit or one byte at a time. Example: RC4

Features

- Faster for continuous data streams
- Used in wireless communication
- Suitable for real-time applications

Modes of Operation in Block Cipher

Block ciphers use different modes to encrypt large amounts of data.

1. ECB (Electronic Code Book): Each plaintext block is encrypted independently. **Formula**

$$C_i = E_K(P_i)$$

Advantages

- Simple
- Fast

Disadvantages

- Less secure
- Identical plaintext blocks produce identical ciphertext blocks

2. CBC (Cipher Block Chaining): Each plaintext block is XORed with previous ciphertext block before encryption. **Formula**

$$C_i = E_K(P_i \oplus C_{i-1})$$

Advantages

- More secure than ECB
- Hides data patterns

Disadvantages

- Slower due to chaining

3. CFB (Cipher Feedback): Converts block cipher into stream cipher. **Features**

- Encrypts smaller units
- Useful in network communication

4. OFB (Output Feedback): Uses previous output for next encryption process. **Features**

- Error propagation is low
- Suitable for noisy channels

5. CTR (Counter Mode): Uses counter values combined with key for encryption. **Features**

- Very fast
- Supports parallel processing
- Highly efficient

Mode	Full Form	Main Trick / Property	Pattern Visibility	Parallel Processing	Error Propagation	IV Needed	Speed
ECB	Electronic Code Book	"Same plaintext → Same ciphertext"	Visible	Yes	Only one block affected	No	Fast
CBC	Cipher Block Chaining	"Each block depends on previous ciphertext"	Hidden	Encryption: No, Decryption: Yes	Error affects current + next block	Yes	Moderate
CFB	Cipher Feedback	"Turns block cipher into stream cipher"	Hidden	No	Error propagates for some bits/blocks	Yes	Moderate
OFB	Output Feedback	"Key stream generated independently"	Hidden	No	No error propagation	Yes	Fast
CTR	Counter Mode	"Uses counter values instead of chaining"	Hidden	Yes	Only affected block changes	Yes (Nonce/Counter)	Very Fast

Advantages

1. Fast encryption and decryption
2. Efficient for large data
3. Requires less computational power
4. Easy implementation

Disadvantages

1. Key distribution problem
2. Less secure for communication over public networks
3. If key is leaked, entire communication becomes insecure

Applications

- File encryption
- Disk encryption
- VPNs
- Secure communication
- Banking systems
- Wireless security

Confusion and Diffusion

- Confusion and Diffusion are two fundamental principles of modern cryptography used to design strong encryption algorithms that resist cryptanalysis and attacks.
- The main goal is to make the relationship between: Plaintext, Ciphertext, Key as complex and unpredictable as possible.
- Modern encryption algorithms such as AES and DES use both confusion and diffusion together.

Confusion

- Confusion is a cryptographic technique that makes the relationship between Ciphertext and Encryption key as complex and hidden as possible.
- The objective of confusion is to make it extremely difficult for an attacker to determine the encryption key even if they know the ciphertext.

Objective of Confusion

- Hide key patterns
- Increase complexity of encryption
- Prevent attackers from guessing the key
- Make ciphertext appear random

How Confusion is Achieved

Confusion is mainly achieved using:

1. Substitution operations
2. Non-linear transformations
3. S-boxes (Substitution boxes)

Example of Confusion

Suppose plaintext is: HELLO

After applying substitutions and key transformations: XQTPZ

An attacker cannot easily determine:

- Which key was used
- Which substitutions were applied

Characteristics of Confusion

- Makes key relationship complex
- Uses substitution techniques
- Produces unpredictable ciphertext
- Increases encryption security

Advantages of Confusion

- Protects encryption keys
- Makes brute-force analysis harder
- Prevents statistical attacks
- Improves security strength

Weaknesses Without Confusion

Without confusion:

- Key patterns become visible
- Cipher becomes predictable
- Attacks become easier

Diffusion

- Diffusion is a cryptographic technique that spreads the influence of one plaintext character over many ciphertext characters. 1 bit change in plaintext significantly affects Cipher text
- The goal is to hide statistical patterns present in the plaintext.

Objective of Diffusion

- Remove plaintext patterns
- Spread changes across ciphertext
- Prevent frequency analysis
- Increase randomness

How Diffusion is Achieved

Diffusion is mainly achieved using:

1. Permutation
2. Transposition
3. Mixing operations

Example of Diffusion

Suppose plaintext is: HELLO

If one letter changes: HELLO → CELLO

A good diffusion mechanism causes many ciphertext characters to change.

Example:

HELLO → QWERT

CELLO → PLMKO

Even a small plaintext change produces a large ciphertext change.

Avalanche Effect: Diffusion creates the Avalanche Effect. A small change in plaintext or key causes a large change in ciphertext.

Example: Changing one bit in plaintext may change: 50% or more ciphertext bits. This makes cryptanalysis extremely difficult.

Characteristics of Diffusion

- Spreads plaintext influence
- Hides statistical properties
- Uses permutation/transposition
- Produces avalanche effect

Advantages of Diffusion

- Prevents frequency analysis
- Increases randomness
- Improves ciphertext complexity
- Enhances security

Weaknesses Without Diffusion

Without diffusion:

- Plaintext patterns remain visible

- Frequency analysis becomes possible
- Cipher becomes weak

Difference Between Confusion and Diffusion

Feature	Confusion	Diffusion
Purpose	Hide relationship between key and ciphertext	Spread plaintext influence across ciphertext
Achieved By	Substitution	Permutation / Transposition
Focus	Key complexity	Plaintext pattern hiding
Main Goal	Make key difficult to find	Remove statistical patterns
Example Technique	S-box	P-box

Importance of Confusion and Diffusion

Together they:

- Strengthen encryption
- Prevent cryptanalysis
- Hide patterns
- Increase randomness
- Improve modern cipher security

Both principles are essential for designing secure block ciphers.

Stream Cipher vs. Block Cipher

- Modern symmetric cryptography is split into two operational designs for handling data: **Stream Ciphers** and **Block Ciphers**. While stream ciphers encrypt data bit-by-bit, block ciphers encrypt fixed-size groups of data.
- To make block ciphers highly secure and efficient, cryptographers frequently rely on an architectural framework known as the **Feistel Cipher Structure**.

Stream Ciphers

A stream cipher encrypts digital data continuously, **one bit or one byte at a time**.

- **How it works:** It uses a secret key to generate a continuous, pseudo-random stream of bits called a **keystream**. This keystream is combined with the plaintext bits, typically using a simple bitwise **XOR** operation.
- **Key Characteristics:**
 - Incredibly fast and requires minimal memory or computing power.
 - Low propagation error (an error in one ciphertext bit usually only corrupts that single bit during decryption).
 - Ideal for real-time, continuous data streams where buffering isn't possible (e.g., live telecommunications, streaming audio/video, or network protocols like RC4 or ChaCha20).

Block Ciphers

A block cipher breaks down the plaintext into **fixed-size blocks** (typically 64, 128, or 256 bits) and encrypts the entire block simultaneously.

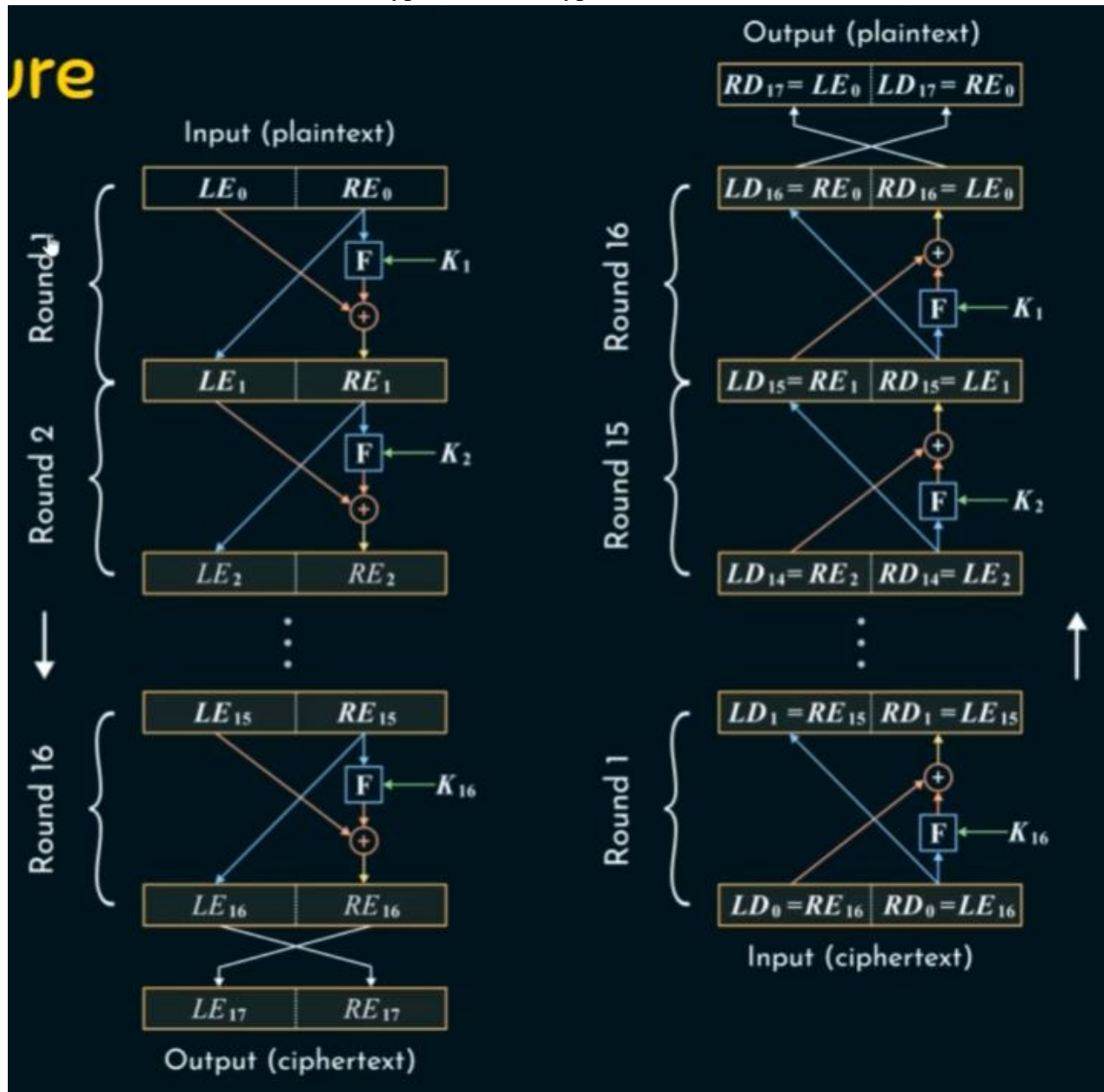
- **How it works:** If a message is smaller than the block size, it uses **padding** to fill the remaining space. If it's larger, it splits the data into chunks and processes them using a specific *Mode of Operation* (like CBC or GCM).
- **Key Characteristics:**
 - Slower than stream ciphers because data must be collected into blocks before processing.
 - Highly secure due to complex internal rounds of math mixing the bits together.
 - Ideal for file storage, database encryption, and secure web traffic (e.g., AES, DES).

Feature	Stream Cipher	Block Cipher
Basic Idea	Encrypts data bit by bit or byte by byte	Encrypts data in fixed-size blocks
Speed	Very fast	Relatively slower
Complexity	Simple design	More complex design
Key Usage	Uses a key stream generated from secret key	Uses same key for each block (with rounds)
Principal Used	Confusion	Both Confusion and Diffusion
Encryption Modes	CFB (Cipher Feedback), OFB (Output Feedback)	ECB (Electronic Codebook), CBC (Cipher Block Chaining)
Decryption Method	XOR operation (same as encryption in most cases)	Reverse of encryption process using same key
Memory Requirement	Low	Higher
Error Propagation	Affects only one bit/byte	Affects entire block
Padding Required	No padding needed	Padding required if data is not multiple of block size
Security Level	Moderate (depends on key stream quality) Very dangerous if key stream reused	High security
Suitable For	Real-time data transmission	File encryption, databases
Latency	Very low	Higher due to block processing
Examples	RC4, Vernam Cipher	AES, DES, Blowfish, 3DES

The Feistel Cipher Structure

Feistel Cipher Structure is a symmetric encryption design model used in many block ciphers. Many modern encryption algorithms are based on Feistel structure. It is a repeatable structural design framework. It serves as the foundation for classic ciphers like **DES (Data Encryption Standard)** and Blowfish.

- Plaintext block is divided into two equal halves
- Multiple rounds of processing are performed
- Same structure is used for encryption and decryption



The Feistel network splits a data block into two equal halves, Left (L) and Right (R), and processes them through multiple rounds.

1. Encryption Process (Left Side of Diagram)

The encryption takes a plaintext block, splits it, and moves downward from Round 1 to Round 16.

Initial Split

The plaintext block is divided into two halves:

- $LE_0 \rightarrow$ Left Encryption initial state
- $RE_0 \rightarrow$ Right Encryption initial state

General Round Execution ($i = 1$ to 16)

For any round i , the new states are computed as:

Left Half Update

The new left half is a direct copy of the previous right half:

$$LE_i = RE_{i-1}$$

Right Half Update

The previous right half RE_{i-1} and round subkey K_i are passed into function F .
The output is XORed with the previous left half:

$$RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$$

Final Swap (After Round 16)

After the 16th round, a final swap is performed to form ciphertext:

$$LE_{17} = RE_{16}$$

$$RE_{17} = LE_{16}$$

2. Decryption Process (Right Side of Diagram)

Decryption takes the ciphertext and moves upward from Round 1 to Round 16.

The same function F is used, but subkeys are applied in reverse order:

$$K_{16} \rightarrow K_1$$

Initial Split (From Ciphertext)

Ciphertext is mapped as:

$$LD_0 = RE_{16}$$

$$RD_0 = LE_{16}$$

General Round Execution ($j = 1$ to 16)

The decryption process mirrors encryption because:

$$A \oplus B \oplus B = A$$

Left Half Update

$$LD_j = RD_{j-1}$$

Right Half Update

$$RD_j = LD_{j-1} \oplus F(RD_{j-1}, K_{17-j})$$

3. Mathematical Proof of Equivalence

To understand why decryption works, compare Decryption Round 1 with Encryption Round 16.

From Encryption Round 16

$$LE_{16} = RE_{15}$$

$$RE_{16} = LE_{15} \oplus F(RE_{15}, K_{16})$$

Decryption Round 1 Input

$$LD_0 = RE_{16}$$

$$RD_0 = LE_{16} = RE_{15}$$

Decryption Round 1

$$LD_1 = RD_0 = RE_{15}$$

$$RD_1 = LD_0 \oplus F(RD_0, K_{16})$$

Substitute values:

$$RD_1 = [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16})$$

Using XOR property:

$$A \oplus B \oplus B = A$$

So:

$$RD_1 = LE_{15}$$

Result After Round 1

$$(LD_1, RD_1) = (RE_{15}, LE_{15})$$

This matches the state before Encryption Round 16, proving that the Feistel structure reverses step by step.

1. Structural Elements: The Math & Mechanics

- a. **XOR ($\oplus$):** The Exclusive OR (XOR) operation is the mathematical backbone of the Feistel structure. It has key properties:

$$A \oplus B \oplus B = A$$

$$A \oplus 0 = A$$

Design Role:

- XOR combines the output of the round function F with one half of the data block.
- It ensures that no information is permanently lost.
- This property makes the cipher reversible during decryption.

- b. **Swap:** At the end of every round, the left and right halves are exchanged:

$$L_i = R_{i-1}$$

Design Role:

- Ensures both halves are processed equally.
- Prevents one half from remaining unchanged.

- Enables full diffusion across rounds.

Without swapping:

- One half would remain unused
- Security would be significantly weaker

2. Core Tuning Parameters

- a. **Block Size:** Defines the size of data processed in one operation. Data is split into two equal halves. **Typical values:**

- 64 bits (e.g., legacy systems like DES)
- 128 bits (modern systems)

Design Impact:

- Small block size $\rightarrow$ repetition patterns $\rightarrow$ birthday/statistical attacks
- Large block size $\rightarrow$ slower performance
- Balanced size ensures security + efficiency

- b. **Key Size:** Length of the master secret key. **Typical values:** 128, 192, 256 bits (modern standards)

Design Impact:

- Determines resistance to brute-force attacks
- Larger key $\rightarrow$ exponentially harder to break
- Directly controls security strength

- c. **Number of Rounds:** Feistel cipher is iterative. Each round applies the same structure with different subkeys. **Typical range:** 16 to 32 rounds

Design Impact:

- Few rounds $\rightarrow$ weak security
- More rounds $\rightarrow$ higher confusion and diffusion
- Sufficient rounds make ciphertext appear random

3. Algorithmic Functions

- a. **Round Function (F):** The round function F is the core security component. It takes one half of the data block and a round subkey to create a complex, pseudo-random bit pattern.

Internal operations:

- Substitution (S-boxes) → Confusion
- Permutation (P-boxes) → Diffusion

Design Role:

- Introduces non-linearity
- Prevents mathematical prediction of output
- Strengthens resistance to cryptanalysis

- b. **Subkey Generation (Key Schedule):** Master This algorithm takes the master user key and slices/shifts/permutates it to generate a unique, smaller subkey (K_i) for each specific round.

$$K_1, K_2, \dots, K_n$$

Process:

- Permutations
- Shifting
- Compression

Design Role:

- Ensures each round uses a different key
- Prevents simple key recovery
- Protects against partial key leakage

4. Performance & Ease of Analysis

- a. **Fast Encryption / Decryption Efficiency Features:** Uses simple operations:

- XOR
- Bit shifts
- Table lookups (S-boxes)

Hardware Advantage:

- Same structure for encryption and decryption
- Only subkeys are reversed
- Saves hardware and memory resources

Design Benefit:

- High speed in software and hardware implementations
- Suitable for real-time systems

- b. **Ease of Analysis:** Feistel structure is easy to analyze mathematically because:

- Clear separation of halves
- Well-defined round structure
- Predictable transformations per round

Security Testing:

- Differential cryptanalysis
- Linear cryptanalysis

Design Benefit:

- Cryptographers can determine required number of rounds
- Helps prove strength of cipher before deployment